

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN
FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



“Interactuar con un sistema operativo UNIX (XV6)”

Asignatura:

Sistemas Operativos

Docente:

MSc. Hugo Manuel Barraza Vizcarra

Estudiantes:

- Jhon William Mamani Condori 2020-119018
- Juan Luis Mamani Mullo 2021-119123

Tacna - Perú

2025

Introducción

Los sistemas operativos constituyen uno de los componentes fundamentales del software de un sistema de cómputo, ya que son los encargados de administrar y coordinar el uso eficiente de los recursos de hardware, tales como el procesador, la memoria principal, los dispositivos de entrada y salida y el sistema de archivos. Para cumplir esta función, el sistema operativo implementa mecanismos que permiten la ejecución concurrente de múltiples procesos, garantizando aislamiento, seguridad y un uso óptimo de los recursos disponibles.

Para lograr una adecuada separación entre los procesos que se ejecutan en el sistema, los sistemas operativos distinguen entre operaciones privilegiadas, gestionadas por el núcleo, y operaciones no privilegiadas, ejecutadas en el espacio de usuario. Esta separación permite un control seguro y eficiente del acceso a los recursos del sistema.

El estudio de los sistemas operativos no solo requiere la comprensión teórica de conceptos como la planificación de procesos, la memoria virtual y las llamadas al sistema, sino también su análisis práctico mediante la interacción con un sistema operativo real. En este contexto, el sistema operativo XV6 representa una herramienta académica idónea, al tratarse de una versión simplificada de UNIX diseñada específicamente para fines educativos, que permite explorar y modificar su núcleo de manera controlada.

El presente trabajo tiene como finalidad extender el sistema operativo XV6 mediante la instrumentación de llamadas al sistema, la implementación de nuevos comandos de usuario y el desarrollo de mecanismos de monitoreo interno, permitiendo analizar su comportamiento y relacionar los resultados obtenidos con los contenidos desarrollados en la Unidad II del curso de Sistemas Operativos.

1. Objetivos

1.1. Objetivo general

Implementar extensiones al sistema operativo XV6 que permitan analizar el funcionamiento de las llamadas al sistema, la planificación de procesos y el uso de los recursos del sistema, relacionando estos mecanismos con los conceptos teóricos desarrollados en la Unidad II del curso de Sistemas Operativos.

1.2. Objetivos específicos

- Instrumentar las llamadas al sistema de XV6 para registrar su ejecución y los parámetros utilizados.
- Implementar nuevos comandos de usuario orientados a la inspección del estado del sistema y de los procesos.
- Desarrollar un contador de invocaciones por llamada al sistema accesible desde el espacio de usuario.
- Evaluar el comportamiento del sistema operativo mediante pruebas prácticas y análisis de resultados.
- Relacionar los resultados obtenidos con los conceptos de planificación, memoria y gestión de procesos estudiados en la Unidad II.

2. Descripción de las modificaciones realizadas (por sección: instrumentación, nuevos comandos, contador de syscalls).

2.1. Instrumentación de llamadas al sistema

2.1.1. Variable global de traza

Archivo: syscall.c

```
int trace_syscalls = 0;
```

La variable global `int trace_syscalls = 0`; se define en el núcleo de XV6 como un **flag de control** que permite habilitar o deshabilitar la instrumentación de llamadas al sistema. Este tipo de variable global es común en XV6 para mantener estado del kernel accesible durante el despacho de syscalls, permitiendo activar mecanismos de monitoreo sin alterar el flujo normal de ejecución (Cox, Kaashoek & Morris, 2014).

2.1.2. Tabla de nombres de syscalls

Archivo: syscall.c

```
static char *syscall_names[] = {
    [SYS_fork]  "fork",
    [SYS_exit]  "exit",
    [SYS_wait]  "wait",
    [SYS_read]  "read",
    [SYS_write] "write",
    [SYS_open]  "open",
    [SYS_close] "close",
    [SYS_kill]  "kill",
    [SYS_exec]  "exec",
    [SYS_getpid] "getpid",
    [SYS_sbrk]  "sbrk",
    [SYS_sleep] "sleep",
    [SYS_uptime] "uptime",
    [SYS_trace] "trace",
    // etc, según tu tabla
};
```

El arreglo `syscall_names` asocia cada **identificador numérico de syscall** con su **nombre simbólico**, permitiendo mostrar en consola el nombre de la llamada al sistema invocada durante el proceso de instrumentación. Este mapeo complementa la tabla de syscalls de XV6 y facilita la depuración y el análisis del flujo de ejecución en el kernel, siguiendo el modelo de despacho de llamadas al sistema descrito en el libro de XV6 (Cox, Kaashoek & Morris, 2014).

2.1.3. Modificar syscall() para instrumentar

Archivo: syscall.c

buscar la funcion

```
void syscall(void)
{
    int num;
    struct proc *curproc = myproc();
    num = curproc->tf->eax;
    ...
}
```

Dentro del if donde se ejecuta la syscall, agregar:

```
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    if (trace_syscalls) {
        cprintf("[syscall] %s\n", syscall_names[num]);
    }
    curproc->tf->eax = syscalls[num]();
} else {
    ...
}
```

Este bloque corresponde al **despacho de llamadas al sistema** en la función `syscall()`. El número de syscall se obtiene desde el *trap frame* del proceso (`curproc->tf->eax`) y se utiliza para indexar la tabla `syscalls`. Cuando el *flag* `trace_syscalls` está activo, el kernel imprime el **nombre de la syscall invocada** antes de ejecutar su manejador. Este mecanismo sigue el flujo estándar de XV6, donde `syscall()` valida el identificador, ejecuta la función asociada y devuelve el resultado al proceso de usuario (Cox, Kaashoek & Morris, 2014).

2.1.4. Syscall del kernel: `sys_trace`

Archivo: `sysproc.c`

Declaración de la variable externa

```
extern int trace_syscalls;
```

agregar al final

```
int
sys_trace(void)
{
    int enable;
```

```
if(argint(0, &enable) < 0)
    return -1;

trace_syscalls = enable;
return 0;
}
```

Este código declara la variable global `trace_syscalls` como externa para su uso en `sysproc.c` e implementa la llamada al sistema `sys_trace()`, la cual recibe un argumento desde el espacio de usuario mediante `argint` y actualiza el estado del *flag* de trazado en el kernel. Este patrón sigue el mecanismo estándar de XV6 para la implementación de syscalls, donde los argumentos son recuperados de forma segura y la lógica privilegiada se ejecuta en el núcleo (Cox, Kaashoek & Morris, 2014).

2.1.5. Registrar syscall trace

Archivo: syscall.h

```
#define SYS_trace 23 // o el número libre que usaste
```

Archivo: syscall.c

agregar arriba

```
extern int sys_trace(void);
```

En la tabla de syscalls

```
[SYS_trace] sys_trace,
```

La constante `#define SYS_trace 23` asigna un **identificador numérico único** a la llamada al sistema `trace`. La declaración `extern int sys_trace(void);` permite enlazar el manejador del kernel con el archivo `syscall.c`, y la entrada `[SYS_trace] = sys_trace` incorpora la syscall en la **tabla de despacho**. En XV6, este mecanismo —identificador, declaración externa y registro en la tabla— es el procedimiento estándar para integrar nuevas llamadas al sistema y permitir que `syscall()` las invoque correctamente (Cox, Kaashoek & Morris, 2014).

2.1.6. Exponer a espacio de usuario

Archivo: user.h

```
int trace(int enable);
```

Archivo: usys.S

```
SYSCALL(trace)
```

La declaración `int trace(int enable);` expone la **syscall** `trace` al **espacio de usuario**, mientras que la macro `SYSCALL(trace)` en `usys.S` genera el *stub* en ensamblador que permite invocar dicha llamada al sistema mediante una trampa al kernel. En XV6, este mecanismo define el enlace estándar entre programas de usuario y el núcleo para la ejecución de llamadas al sistema (Cox, Kaashoek & Morris, 2014).

2.1.7. Comando de usuario `trace`

Archivo nuevo: `trace.c`

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
    if(argc < 2){
        printf(1, "Uso: trace <0|1>\n");
        exit();
    }

    int enable = atoi(argv[1]);
    trace(enable);

    exit();
}
```

Este archivo implementa el **comando de usuario** `trace`, el cual permite activar o desactivar el trazado de llamadas al sistema desde el espacio de usuario. El programa recibe un argumento (`0` o `1`), lo convierte a entero y llama a la syscall `trace(enable)`, estableciendo una comunicación controlada con el kernel. En XV6, este patrón —comando en espacio de usuario que invoca una syscall definida en

`user.h`— es el mecanismo estándar para interactuar con funcionalidades del núcleo (Cox, Kaashoek & Morris, 2014).

2.1.8. Comando de usuario `trace`

Archivo: Makefile

```
_trace\
```

La entrada `_trace` se agrega al **Makefile** de XV6 para incluir el nuevo comando de usuario `trace` en la lista de programas que se compilan y se incorporan a la imagen del sistema. En XV6, todos los programas de usuario deben declararse explícitamente en el Makefile para que formen parte del sistema de archivos inicial y puedan ejecutarse desde la consola, siguiendo el modelo estándar de construcción del sistema (Cox, Kaashoek & Morris, 2014).

2.1.9. Prueba

```
make clean
make qemu

//en qemu
$ trace 1
$ ls
//Apagar traza
$ trace 0
```

Esta secuencia corresponde a la **validación funcional** del mecanismo de instrumentación implementado. Tras recompilar y ejecutar XV6 (`make clean`, `make qemu`), el comando de usuario `trace 1` habilita el trazado de llamadas al sistema; al ejecutar `ls`, el kernel muestra en consola las syscalls invocadas. Finalmente, `trace 0` desactiva la traza, verificando que el control mediante *flag* funciona correctamente. Este flujo es coherente con el modelo de construcción, ejecución y depuración de XV6, donde los comandos de usuario interactúan con el kernel a través de llamadas al sistema (Cox, Kaashoek & Morris, 2014).

2.2. Comandos de usuario

2.2.1. Comando uptime2

2.2.1.1. Nueva syscall: getuptimeinfo

Archivo: sysproc.c

```
extern int ncontext;

//agregar funcion
int
sys_getuptimeinfo(void)
{
    int *addr;

    if(argptr(0, (char**)&addr, sizeof(int) * 5) < 0)
        return -1;

    int running = 0, runnable = 0, sleeping = 0;
    struct proc *p;

    addr[0] = ticks;

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        switch(p->state){
            case RUNNING: running++; break;
            case RUNNABLE: runnable++; break;
            case SLEEPING: sleeping++; break;
        }
    }

    addr[1] = running;
    addr[2] = runnable;
    addr[3] = sleeping;
    addr[4] = ncontext;

    return 0;
}
```

Este bloque implementa la llamada al sistema **sys_getuptimeinfo()**, la cual permite **exponer información interna del kernel al espacio de usuario** de forma controlada. Mediante **argptr**, se valida un puntero donde el kernel escribe un conjunto de métricas: el tiempo de ejecución del sistema (**ticks**), la cantidad de procesos en estado **RUNNING**, **RUNNABLE** y **SLEEPING**, y el número de **cambios de contexto** (**ncontext**). El recorrido de la tabla de procesos (**ptable**) y la

clasificación por estados siguen el diseño del planificador de XV6, permitiendo analizar planificación y gestión de procesos, tal como se describe en el libro oficial del sistema (Cox, Kaashoek & Morris, 2014).

2.2.1.2. Registrar `getuptimeinfo`

```
//Archivo: syscall.h
#define SYS_getuptimeinfo 24

//Archivo: syscall.c
extern int sys_getuptimeinfo(void);

//En la tabla
[SYS_getuptimeinfo] sys_getuptimeinfo,

//Archivo: user.h
int getuptimeinfo(int *);

//Archivo: usys.S
SYSCALL(getuptimeinfo)
```

Estas modificaciones integran la nueva llamada al sistema **getuptimeinfo** en XV6. La constante `#define SYS_getuptimeinfo 24` asigna su **identificador numérico**; la declaración `extern int sys_getuptimeinfo(void);` y su registro en la tabla `[SYS_getuptimeinfo] = sys_getuptimeinfo` permiten que el despachador `syscall()` invoque el manejador del kernel. Asimismo, la declaración `int getuptimeinfo(int *);` en `user.h` y la macro `SYSCALL(getuptimeinfo)` en `usys.S` exponen la syscall al espacio de usuario mediante el *stub* estándar. Este flujo corresponde al mecanismo canónico de incorporación de nuevas syscalls en XV6 (Cox, Kaashoek & Morris, 2014).

2.2.1.3. Comando `uptime2`

Archivo: `uptime2.c`

```
#include "types.h"
#include "stat.h"
#include "user.h"
```

```

int
main(void)
{
    int info[5];

    if(getuptimeinfo(info) < 0){
        printf(1, "Error: getuptimeinfo\n");
        exit();
    }

    int ticks    = info[0];
    int running  = info[1];
    int runnable = info[2];
    int sleeping = info[3];
    int ctx      = info[4];

    printf(1, "Tiempo desde arranque: %d ticks (%d
segundos)\n", ticks, ticks/100);
    printf(1, "Procesos RUNNING : %d\n", running);
    printf(1, "Procesos RUNNABLE: %d\n", runnable);
    printf(1, "Procesos SLEEPING: %d\n", sleeping);
    printf(1, "Cambios de contexto: %d\n", ctx);

    exit();
}

```

Este archivo implementa un **comando de usuario** que invoca la syscall **getuptimeinfo** para obtener información del estado del sistema desde el kernel. Los datos devueltos incluyen el tiempo de ejecución del sistema en *ticks*, el número de procesos en estados **RUNNING**, **RUNNABLE** y **SLEEPING**, así como la cantidad de **cambios de contexto**. Este enfoque sigue el modelo de XV6 para exponer información interna del núcleo al espacio de usuario mediante llamadas al sistema, permitiendo analizar aspectos de planificación y gestión de procesos (Cox, Kaashoek & Morris, 2014).

2.2.1.4. Registrar uptime2 en Makefile

Archivo: Makefile

En UPROGS AGREGAR

```
_uptime2\
```

2.2.1.5. Programa auxiliar spin para generar RUNNABLE

Archivo crear: spin.c

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    for(;;)
        ; // bucle infinito
    exit();
}
```

Este programa de usuario implementa un **bucle infinito** que mantiene al proceso en ejecución permanente, con el objetivo de **forzar actividad del planificador** y generar cambios de contexto en el sistema. En XV6, este tipo de procesos es comúnmente utilizado con fines educativos para observar el comportamiento del planificador, los estados de los procesos y el impacto en la gestión de CPU, tal como se describe en el análisis de planificación y procesos del sistema (Cox, Kaashoek & Morris, 2014).

Agregar spin al Makefile:

```
_spin\
```

La entrada `_spin` se agrega al **Makefile** para incluir el programa de usuario `spin` en la imagen de XV6. Al declararlo explícitamente, el sistema lo compila y lo incorpora al sistema de archivos inicial, permitiendo su ejecución desde la consola. En XV6, este es el procedimiento estándar para integrar nuevos programas de usuario al proceso de construcción del sistema (Cox, Kaashoek & Morris, 2014).

2.2.1.6. Pruebas

ejecutar

```
$ uptime2
$ spin &
$ uptime2
$ kill <pid_de_spin>
```

2.2.2. Comando psmen (inspección de procesos)

2.2.2.1. Estructura struct uproc en el kernel

Archivo: proc.h

```
struct uproc {
    int pid;
    int state;
    int sz;
    char name[16];
};
```

La estructura `struct uproc` define un **formato simplificado de información de procesos** destinado a ser compartido con el espacio de usuario. Contiene datos básicos como el identificador del proceso (`pid`), su estado, el tamaño de memoria (`sz`) y el nombre del proceso. En XV6, este tipo de estructuras se utiliza para exponer información del kernel de manera controlada, manteniendo el aislamiento entre el núcleo y los programas de usuario, conforme al diseño del sistema descrito en el libro oficial (Cox, Kaashoek & Morris, 2014).

2.2.2.2. Syscall getps

Archivo: sysproc.c

```
int
sys_getps(void)
{
    struct uproc *up;
    int max;

    if(argptr(0, (char**)&up, sizeof(struct uproc) * 64) < 0)
        return -1;

    if(argint(1, &max) < 0)
        return -1;
```

```

int count = 0;
struct proc *p;

for(p = ptable.proc; p < &ptable.proc[NPROC] && count <
max; p++){
    if(p->state == UNUSED) continue;

    up[count].pid = p->pid;
    up[count].state = p->state;
    up[count].sz = p->sz;
    safestrcpy(up[count].name, p->name, 16);
    count++;
}

return count;
}

```

La función **sys_getps()** implementa una llamada al sistema que permite **obtener información de los procesos activos** y transferirla de forma segura al espacio de usuario. Utilizando **argptr** y **argint**, el kernel valida un arreglo de estructuras **uproc** y un límite máximo de procesos a retornar. Luego recorre la **tabla de procesos (ptable)**, copiando para cada proceso válido su PID, estado, tamaño de memoria y nombre. Este mecanismo sigue el diseño de XV6 para exponer información del kernel de manera controlada, preservando el aislamiento entre núcleo y espacio de usuario, y es coherente con el modelo de gestión de procesos y planificación descrito en el libro oficial del sistema (Cox, Kaashoek & Morris, 2014).

2.2.2.3. Registrar getps

Archivo: syscall.h

```

#define SYS_getps 25

//Archivo: syscall.c
extern int sys_getps(void);
[SYS_getps] sys_getps,

//Archivo: user.h
struct uproc;           // forward declaration
int getps(struct uproc*, int);

```

```
//Archivo: usys.S  
SYSCALL(getps)
```

Estas modificaciones integran la nueva llamada al sistema `getps` en XV6. La constante `#define SYS_getps 25` asigna su identificador numérico; la declaración `extern int sys_getps(void);` y su registro en la tabla `[SYS_getps] = sys_getps` permiten que el despachador de syscalls invoque el manejador del kernel. Asimismo, la declaración de la estructura `uproc` y la función `int getps(struct uproc*, int);` en `user.h`, junto con la macro `SYSCALL(getps)` en `usys.S`, exponen la syscall al espacio de usuario mediante el mecanismo estándar de invocación. Este flujo corresponde al procedimiento canónico para añadir nuevas llamadas al sistema en XV6 (Cox, Kaashoek & Morris, 2014).

2.2.2.4. Comando de usuario psmen

Archivo crear: psmen.c

```
#include "types.h"  
#include "stat.h"  
#include "user.h"  
  
// Versión de struct uproc para espacio de usuario  
struct uproc {  
    int pid;  
    int state;  
    int sz;  
    char name[16];  
};  
  
char *states[] = {  
    "UNUSED",  
    "EMBRYO",  
    "SLEEPING",  
    "RUNNABLE",  
    "RUNNING",  
    "ZOMBIE"  
};
```

```

int
main(void)
{
    struct uproc ps[64];
    int n = getps(ps, 64);

    printf(1, "PID  Estado   Mem(Bytes)  Nombre\n");

    for(int i = 0; i < n; i++){
        printf(1, "%d   %s   %d   %s\n",
            ps[i].pid,
            states[ps[i].state],
            ps[i].sz,
            ps[i].name);
    }

    exit();
}

```

Este archivo implementa un **comando de usuario tipo ps** que obtiene y muestra información de procesos invocando la syscall **getps(ps, 64)**. Los datos retornados por el kernel (PID, estado, tamaño de memoria y nombre) se almacenan en un arreglo de estructuras **uproc** y luego se presentan en formato de tabla, traduciendo el estado numérico a una etiqueta legible. Este diseño sigue el enfoque de XV6, donde los programas de usuario consultan información del sistema mediante llamadas al sistema y estructuras de datos simples, reforzando el análisis de gestión de procesos y estados descrito en el libro oficial (Cox, Kaashoek & Morris, 2014).

2.2.2.5. Registrar psmen en Makefile

Archivo: Makefile

```
_psmen\
```

La entrada **_psmem** se añade al **Makefile** para incluir el nuevo comando de usuario **psmem** en la imagen de XV6. Al declararlo explícitamente, el programa se compila y se incorpora al sistema de archivos inicial, permitiendo su ejecución desde la consola. En XV6, este es el mecanismo

estándar para integrar programas de usuario durante el proceso de construcción del sistema (Cox, Kaashoek & Morris, 2014).

2.2.2.6. Pruebas

```
$ psmen
```

2.3. Contador de invocaciones por syscall

2.3.1. Arreglo de contadores en el kernel

Archivo: syscall.c

```
int syscall_count[NELEM(syscalls)];
```

El arreglo `int syscall_count[NELEM(syscalls)];` define una **estructura de contadores en el núcleo** para registrar el número de invocaciones de cada llamada al sistema. Cada posición del arreglo corresponde al identificador de una syscall, permitiendo incrementar el contador asociado durante el despacho en `syscall()`. Este enfoque aprovecha la tabla de syscalls de XV6 y sigue el diseño simple y eficiente del sistema para mantener métricas internas del kernel con fines de análisis y monitoreo (Cox, Kaashoek & Morris, 2014).

2.3.2. Incrementar el contador en syscall()

En la función `syscall(void)`, dentro del if que valida num:

```
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {  
    syscall_count[num]++;           // contador  
    curproc->tf->eax = syscalls[num](); // ejecutar syscall  
} else {  
    ...  
}
```

Este bloque incrementa el **contador de invocaciones por syscall** antes de ejecutar su manejador. Al validar el identificador (`num`) y acceder a la tabla `syscalls`, se incrementa `syscall_count[num]` y luego se ejecuta la función correspondiente, devolviendo el resultado al proceso de usuario. Este enfoque se integra naturalmente en el flujo estándar de despacho de `syscall()` en XV6, permitiendo recolectar métricas internas sin alterar la semántica de ejecución de las llamadas al sistema (Cox, Kaashoek & Morris, 2014).

2.3.3. Syscall getsyscount en el kernel

Archivo: sysproc.c

```
extern int syscall_count[];

int
sys_getsyscount(void)
{
    int *buf;
    int size;

    if(argptr(0, (char**)&buf, sizeof(int) * 64) < 0)
        return -1;

    if(argint(1, &size) < 0)
        return -1;

    for(int i = 0; i < size; i++)
        buf[i] = syscall_count[i];

    return 0;
}

int
sys_getsyscount(void)
{
    int *buf;
    int size;

    if(argptr(0, (char**)&buf, sizeof(int) * 64) < 0)
        return -1;

    if(argint(1, &size) < 0)
        return -1;

    for(int i = 0; i < size; i++)
        buf[i] = syscall_count[i];

    return 0;
}
```

La función **sys_getsyscount()** implementa una llamada al sistema que permite **consultar desde el espacio de usuario los contadores de invocaciones de syscalls** almacenados en el kernel. Mediante **argptr** y **argint**, se valida un buffer de enteros y su tamaño, y luego se copian los

valores del arreglo `syscall_count` hacia dicho buffer. Este mecanismo sigue el modelo de XV6 para exponer información interna del núcleo de forma controlada y segura, utilizando estructuras simples y validación de argumentos durante el despacho de llamadas al sistema (Cox, Kaashoek & Morris, 2014).

2.3.4. Registrar getsyscount

Archivo agregar: syscall.h

```
#define SYS_getsyscount 26
//Archivo: syscall.c
extern int sys_getsyscount(void);
[SYS_getsyscount] sys_getsyscount,

//Archivo agregar: user.h
int getsyscount(int *buf, int size);

//Archivo: usys.S
SYSCALL(getsyscount)
```

Estas modificaciones integran la llamada al sistema `getsyscount` en XV6. La constante `#define SYS_getsyscount 26` asigna su **identificador numérico**, mientras que la declaración `extern int sys_getsyscount(void);` y su registro en la tabla `[SYS_getsyscount] = sys_getsyscount` permiten que el despachador `syscall()` invoque el manejador del kernel. Asimismo, la declaración `int getsyscount(int *buf, int size);` en `user.h` y la macro `SYSCALL(getsyscount)` en `usys.S` exponen esta funcionalidad al espacio de usuario mediante el mecanismo estándar de invocación de syscalls en XV6, conforme al diseño descrito en el libro oficial (Cox, Kaashoek & Morris, 2014).

2.3.5. Comando de usuario syscount

Archivo: syscount.c

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
```

```

main(int argc, char *argv[])
{
    int buf[64];

    if(getsyscount(buf, 64) < 0){
        printf(1, "Error: getsyscount\n");
        exit();
    }

    if(argc == 2){
        int id = atoi(argv[1]);
        printf(1, "Syscall %d fue invocada %d veces\n", id, buf[id]);
        exit();
    }

    printf(1, "Resumen de invocaciones:\n");
    for(int i = 0; i < 64; i++){
        if(buf[i] > 0)
            printf(1, "Syscall %d: %d veces\n", i, buf[i]);
    }

    exit();
}

```

Este archivo implementa un **comando de usuario** que consulta el contador de invocaciones de syscalls mediante la llamada al sistema `getsyscount(buf, 64)`. Si se proporciona un parámetro, muestra el número de invocaciones de una syscall específica; si no, presenta un resumen de todas las syscalls que registraron actividad. Este patrón —programa de usuario que obtiene información del kernel a través de una syscall y la presenta de forma legible— es consistente con el enfoque de XV6 para instrumentación y análisis del sistema mediante interfaces simples entre espacio de usuario y núcleo (Cox, Kaashoek & Morris, 2014).

2.3.6. Registrar syscount en el Makefile

Archivo: Makefile

```

_syscount\

```

La entrada `_syscount` se agrega al **Makefile de XV6** para incluir el comando de usuario `syscount` en la imagen del sistema. De esta manera, el programa se compila y se integra al sistema de archivos inicial, permitiendo su ejecución desde la consola. En XV6, este es el

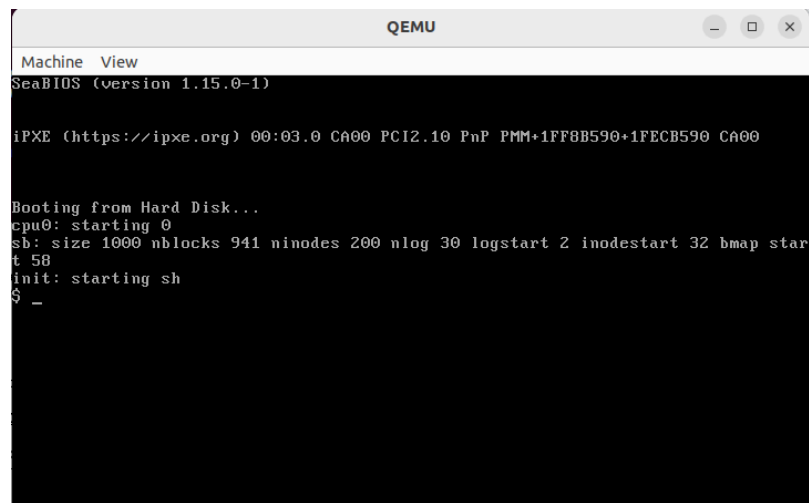
procedimiento estándar para incorporar nuevos programas de usuario al proceso de construcción del sistema operativo (Cox, Kaashoek & Morris, 2014).

2.3.7. Pruebas

```
$ syscount  
  
$ syscount 5  
Syscall 5 fue invocada X veces
```

3. Resultados de pruebas

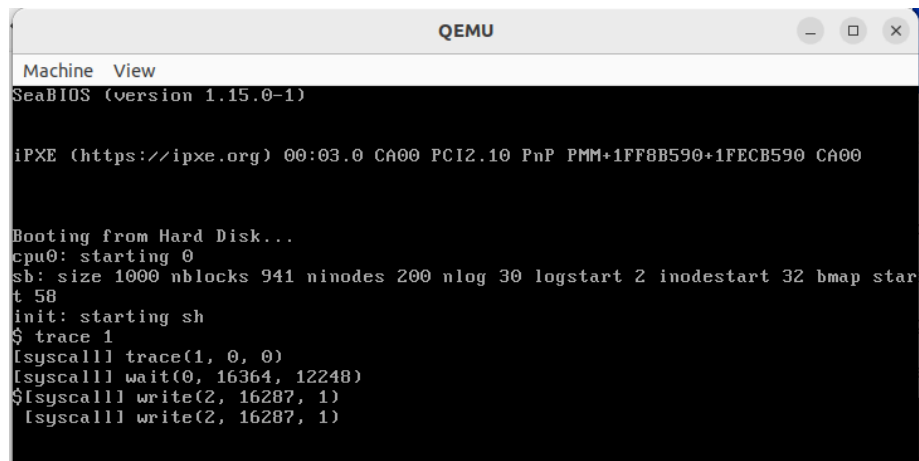
3.1. Arranque del sistema XV6 en QEMU



```
Machine View  
SeaBIOS (version 1.15.0-1)  
  
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00  
  
Booting from Hard Disk...  
cpu0: starting 0  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58  
init: starting sh  
$ _
```

En esta captura se puede observar el arranque correcto del sistema XV6 dentro del emulador QEMU. Confirmando que el entorno está correctamente compilado y listo para poder ejecutar los comandos implementados.

3.2. Ejecución de comandos con traza activada (Entregable 1)

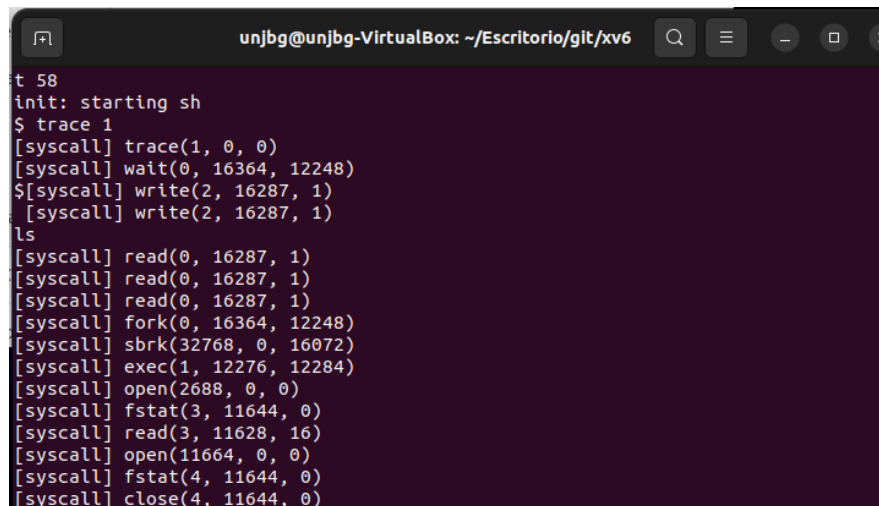


```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

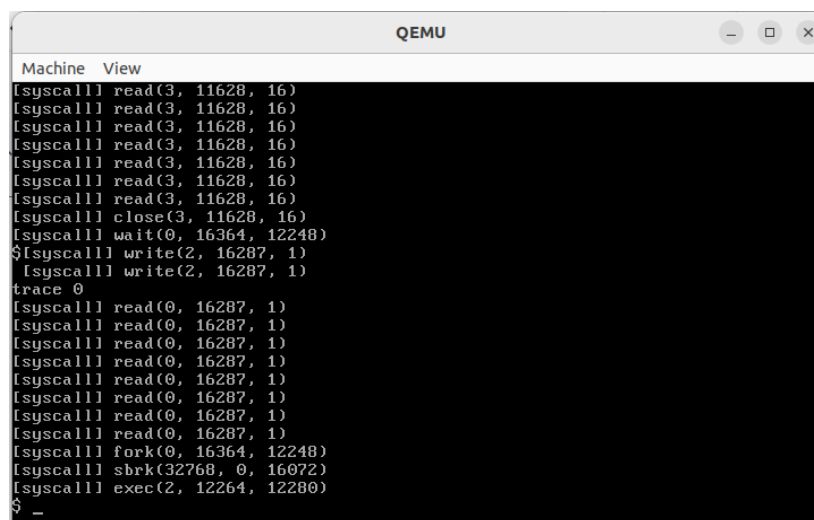
Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ trace 1
[syscall] trace(1, 0, 0)
[syscall] wait(0, 16364, 12248)
[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
```

En la captura se muestra cómo, al ejecutar el comando `trace 1`, se habilita la impresión de las llamadas al sistema en la consola.



```
unjbg@unjbg-VirtualBox: ~/Escritorio/git/xv6
t 58
init: starting sh
$ trace 1
[syscall] trace(1, 0, 0)
[syscall] wait(0, 16364, 12248)
[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
ls
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] fork(0, 16364, 12248)
[syscall] sbrk(32768, 0, 16072)
[syscall] exec(1, 12276, 12284)
[syscall] open(2688, 0, 0)
[syscall] fstat(3, 11644, 0)
[syscall] read(3, 11628, 16)
[syscall] open(11664, 0, 0)
[syscall] fstat(4, 11644, 0)
[syscall] close(4, 11644, 0)
```

Luego, al ejecutar el `ls`, el kernel registra syscalls como `open`, `read`, `write`, `close`, etc. Y los estará mostrando a cada uno con sus respectivos parámetros.

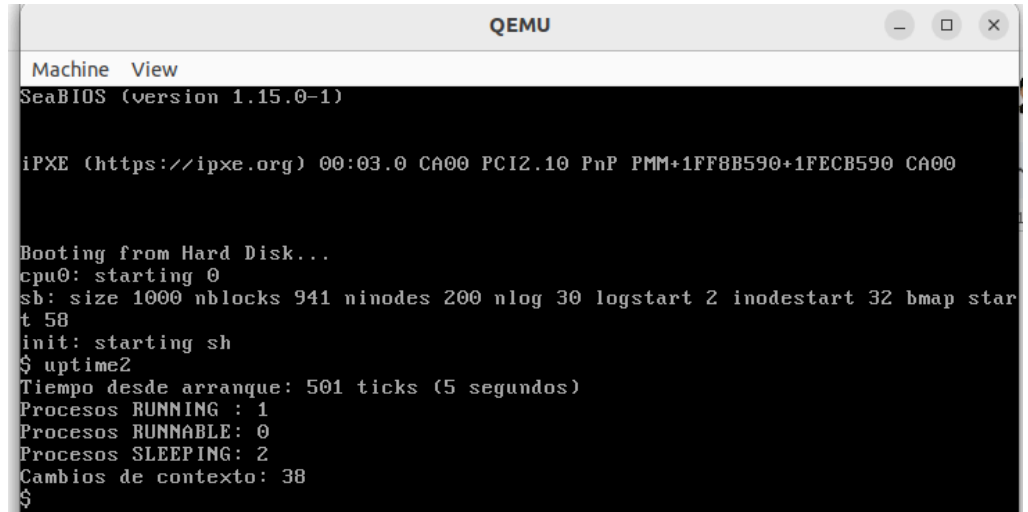


```
QEMU
Machine View
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] close(3, 11628, 16)
[syscall] wait(0, 16364, 12248)
[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
trace 0
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] fork(0, 16364, 12248)
[syscall] sbrk(32768, 0, 16072)
[syscall] exec(2, 12264, 12280)
$ -
```

Finalmente, con `trace 0`, la traza se desactiva, demostrando que el mecanismo funciona correctamente.

3.3. Resultados de la implementación del Entregable 2 (uptime2 y psmen)

3.3.1. Ejecución del comando `uptime2`



```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ uptime2
Tiempo desde arranque: 501 ticks (5 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 38
$
```

La captura muestra el comando `uptime2`, que obtiene información extendida del sistema mediante la syscall `getuptimeinfo`. Se visualizan los ticks desde el arranque, el tiempo total en segundos, el número de procesos RUNNING, RUNNABLE y SLEEPING, así como la cantidad de cambios de contexto. Esta salida confirma que la extracción de datos desde el kernel funciona correctamente.

3.3.2. Ejecución de `spin &` seguido de `uptime2`

```
QEMU - Press Ctrl+Alt+G to release grab
Machine View
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ uptime2
Tiempo desde arranque: 501 ticks (5 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 38
$ spin &
$ uptime2
Tiempo desde arranque: 41052 ticks (410 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 1
Procesos SLEEPING: 2
Cambios de contexto: 703
$ -
```

Aquí se muestra cómo, tras ejecutar **spin &**, aparece un proceso adicional en estado RUNNABLE debido a que **spin** ejecuta un bucle infinito y siempre solicita CPU. Al volver a ejecutar **uptime2**, el contador de procesos RUNNABLE aumenta, demostrando que la syscall detecta correctamente los cambios en el estado de los procesos.

3.3.3. Ejecución del **psmen**

```
Cambios de contexto: 703
$ psmen
PID    Estado      Mem(Bytes)  Nombre
1      SLEEPING     12288       init
2      SLEEPING     16384       sh
7      RUNNING     12288       psmen
5      RUNNABLE     12288       spin
$
```

La captura presenta la salida del comando **psmen**, el cual utiliza la syscall **getps** para listar todos los procesos en ejecución. Se muestran sus PID, estado, memoria utilizada y nombre. Se observa que procesos como **sh** e **init** están en SLEEPING, el comando **psmen** aparece como RUNNING, y **spin** se encuentra en RUNNABLE, reflejando el estado real del scheduler.

3.3.4. Eliminación de **spin &** seguido de la ejecución de **uptime2** y **psmen**


```

$ kill 5
zombie!
$ uptime2
Tiempo desde arranque: 50755 ticks (507 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 9840
$ psmen
PID      Estado      Mem(Bytes)  Nombre
1        SLEEPING    12288       init
2        SLEEPING    16384       sh
10       RUNNING    12288       psmen
$

```

En la captura presenta la eliminación del proceso `spin` mediante su PID y haciendo uso del comando `kill <pid>` esto para poder liberar el consumo de cpu que está generando y luego verificamos con `psmen` para ver que fue eliminado.

3.4. Resultados de la implementación del Entregable 3 (syscount)

3.4.1. Contadores globales de syscalls usando `syscount`

```

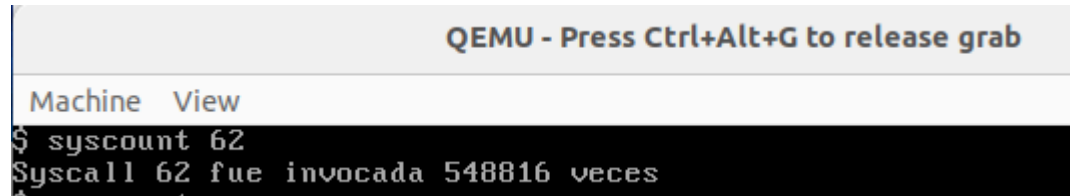
$ syscount
Listado de invocaciones:
Syscall 1: 14 veces
Syscall 2: 11 veces
Syscall 3: 14 veces
Syscall 5: 102 veces
Syscall 6: 1 veces
Syscall 7: 14 veces
Syscall 10: 2 veces
Syscall 12: 12 veces
Syscall 15: 2 veces
Syscall 16: 1308 veces
Syscall 21: 1 veces
Syscall 23: 3 veces
Syscall 24: 2 veces
Syscall 25: 5 veces
Syscall 32: 70205 veces
Syscall 56: 548789 veces
Syscall 58: 548798 veces
Syscall 60: 548807 veces
Syscall 62: 548816 veces
$ -

```

Esta captura muestra el funcionamiento del comando `syscount`, que consulta al kernel mediante la syscall `getsyscount` para obtener el número de invocaciones de cada llamada al sistema. La salida refleja correctamente cuáles syscalls se han utilizado desde el inicio del

sistema y cuántas veces, demostrando que el contador interno funciona y se actualiza con cada llamada real.

3.4.2. Consulta individual



```
QEMU - Press Ctrl+Alt+G to release grab
Machine View
$ syscount 62
Syscall 62 fue invocada 548816 veces
```

En esta imagen se aprecia el uso de `syscount <id>`, donde se obtiene el número específico de invocaciones de una syscall concreta.

4. Conclusiones

La implementación de las funcionalidades añadidas al sistema operativo XV6 permitió comprender de manera práctica cómo se gestionan los procesos, las llamadas al sistema y la planificación dentro del kernel. A través de las modificaciones realizadas, fue posible observar el flujo interno de ejecución de una syscall, el funcionamiento de la tabla de procesos y la transición entre los distintos estados (RUNNING, RUNNABLE y SLEEPING).

Asimismo, la creación de comandos personalizados y nuevas llamadas al sistema hizo evidente la relación entre el espacio de usuario y el espacio del kernel, reforzando conceptos fundamentales como la comunicación mediante trampas, la estructura del scheduler y el manejo de estadísticas del sistema.

En conjunto, el trabajo permitió consolidar conocimientos esenciales sobre el ciclo de vida de los procesos, la organización interna del kernel y las bases del diseño de un sistema operativo, logrando una visión más completa y aplicada de los temas desarrollados en la Unidad II.

5. Referencias bibliográficas

Cox, R., Kaashoek, M. F., & Morris, R. (2023). *xv6: a simple, Unix-like teaching operating system* (Rev. 3). Massachusetts Institute of Technology. Recuperado de <https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf>

6. Anexos

https://github.com/jwmamanic/PRODUCTO_UII_SO.git